

Connector Game Client API

Introduction

This document describes API exposed by *Connector* to the *Game Client*.

The key words `MUST`, `MUST NOT`, `REQUIRED`, `SHALL`, `SHALL NOT`, `SHOULD`, `SHOULD NOT`, `RECOMMENDED`, `MAY`, and `OPTIONAL` in this document are to be interpreted as described in [RFC 2119](#).

Installation

To start using the library you `MUST` load it via `<script>` tag.

```
<script src="/slotify/connector/connector.js"></script>
```

Once it is loaded, you can initialize it

```
const settings = {provider: "slotProvider"};
const callbacks = {mute: () => console.info("mute sounds")};
const theme = {backgroundColor: "white"};
const features = {mute: true, payable: true};

const connector = await window.connector.create(settings, callbacks, theme, features);
```

TypeScript

To use types install the library `@slotify/connector`. To have access to it you need to request private key. One of the ways to configure it is to create `.npmrc` file with the following content where `XXX` should be replaced with private key.

```
//npm.pkg.github.com/:_authToken=XXX
@slotify:registry=https://npm.pkg.github.com/
```

Once installed, add type reference i.e. in `connector.d.ts`

```
///<reference types="@slotify/connector" />
```

Important: DO NOT import code from npm library. It should be used only for types and the code should be loaded via `<script>` tag.

Settings

All fields are `OPTIONAL`. If they are not explicitly provided in the object then the library tries to read it from URL parameters of the same name.

name	type	example	description
provider	string	slotProvider	Name of the provider
wallet	string	someWallet	Name of the wallet

name	type	example	description
operator	string	someOperator	Name of the operator
game	string	myGame	Name of the game
key	string	sadiojsad	Player authentication key (if empty it generates random one). On demo wallet it represents player id
language	string	en	Player's language
server	string	http://server.com	URL to the backend
realityCheckInterval	number	30	Interval in minutes to show the reality check window
realityCheckElapsed	number	7	Initial offset in minutes of the Reality Check interval

Callbacks

Callbacks `freezeBet`, `unfreezeBet`, `stopAutoplay`, `balanceChanged` are used by in game promo tools UI. The rest is typically required by some custom integrations.

name	type	example	description
mute	function	() => null	Mute the sound
unmute	function	() => null	Unmute the sound
turboToggle	function	(value) => null	Toggle turbo capability
paytableToggle	function	(value) => null	Toggle payable capability
helpToggle	function	(value) => null	Toggle help capability
aboutToggle	function	(value) => null	Toggle about capability
stopAutoplay	function	() => null	Stops the autoplay if it's running
play	function	(action, bet, cheat) => null	Triggers play
freezeBet	function	(bet) => null	Freezes bet panel on specific value (for promotional free bets)
unfreezeBet	function	() => null	Unfreezes bet panel
popupOpened	function	(count:number) => null	Popup opened, count is number of popups in the stack
popupClosed	function	(count:number) => null	Popup closed, count is number of popups in the stack
openGameHistory	function	() => null	Open game history window (i.e. by calling <code>connector.gameHistory(...)</code>)
formatCurrency	function	(value: number, currency?: string, language?: string, currencyDecimals?: number) => string	Use custom currency formatting (if not added we will use our own)
freeze	function	() => null	Freezes the game (blocks the screen)

name	type	example	description
<code>unfreeze</code>	<code>function</code>	<code>() => null</code>	Unfreezes the game (unblocks the screen)
<code>balanceChanged</code>	<code>function</code>	<code>(balance:number) => null</code>	Informs about balance changes outside the game i.e. prize drop won

Theme

name	type	example	description
<code>fontFamily</code>	<code>string</code>	<code>Helvetica</code>	Font family
<code>backgroundColor</code>	<code>string</code>	<code>black</code>	Background color
<code>primaryColor</code>	<code>string</code>	<code>#rgb(20, 20, 20)</code>	Primary color
<code>secondaryColor</code>	<code>string</code>	<code>#112233</code>	Secondary color
<code>primaryTextColor</code>	<code>string</code>	<code>red</code>	Primary text color
<code>secondaryTextColor</code>	<code>string</code>	<code>pink</code>	Secondary text color
<code>overlayBackgroundColor</code>	<code>string</code>	<code>pink</code>	Overlay background color

Features

name	type	example	description
<code>mute</code>	<code>boolean</code>	<code>true</code>	Game has mute capabilities
<code>turbo</code>	<code>boolean</code>	<code>true</code>	Game has turbo capabilities
<code>paytable</code>	<code>boolean</code>	<code>true</code>	Game has payable capabilities
<code>help</code>	<code>boolean</code>	<code>true</code>	Game has help capabilities
<code>about</code>	<code>boolean</code>	<code>true</code>	Game has about capabilities

Errors

In case of error each method `SHOULD` throw an `Error`. Connector also takes care of displaying the error popup.

Example:

```
try {
  const {balance, currency} = await connector.authenticate();
} catch (e) {
  console.error("Authorisation failed", e.message);
}
```

Translations

Connectors uses [BCP 47](#) standard and takes care of mapping any locale from underscore to dash, example: `zh_Hans` -> `zh-Hans` .

API

Method: `authenticate` Async: YES Required: YES

```
const {balance, currency} = await connector.authenticate();
```

Authenticates a *Player* and starts a new session.

Returns

Name	Type	Required	Example	Description
<code>balance</code>	<code>number</code>	REQUIRED	<code>123.45</code>	Balance in <i>Player's</i> currency
<code>currency</code>	<code>string</code>	REQUIRED	<code>eur</code>	<i>Player's</i> currency (internal platform code)
<code>currencySymbol</code>	<code>string</code>	REQUIRED	<code>eur</code>	<i>Player's</i> currency (to display to the player)
<code>currencyDecimals</code>	<code>number</code>	REQUIRED	<code>2</code>	<i>Player's</i> currency decimal places
<code>jurisdiction</code>	<code>string</code>	OPTIONAL	<code>mt</code>	<i>Player's</i> jurisdiction
<code>playerId</code>	<code>string</code>	REQUIRED	UUID	<i>Player's</i> id on the platform
<code>nickname</code>	<code>string</code>	OPTIONAL	<code>my nickname</code>	<i>Player's</i> nickname

Method: `play` Async: YES Required: YES

```
const {wager, balance} = await connector.play(action, bet, params, cheat, complete, asyncWin, parallelRound);
```

Creates a new wager.

Arguments

Name	Type		Example	Description
<code>action</code>	<code>string</code>	REQUIRED	<code>gamble</code>	Action
<code>bet</code>	<code>number</code>	OPTIONAL	<code>2.58</code>	Cash bet in player's currency. It <code>SHOULD NOT</code> have more then 2 decimal places, otherwise it will be rounded using bankers rounding. Required only in the initial play
<code>params</code>	<code>any</code>	OPTIONAL	<code>{myValue:123}</code>	Extra params passed to the game server
<code>cheat</code>	<code>string</code>	OPTIONAL	<code>bonus</code>	Cheat for forcing game outcome. Available only in <code>development</code> mode

Name	Type		Example	Description
<code>complete</code>	<code>boolean</code>	OPTIONAL	<code>false</code>	If <code>true</code> , it automatically makes <code>complete()</code> if possible (there is no next action) and makes it return balance after win
<code>asyncWin</code>	<code>boolean</code>	OPTIONAL	<code>true</code>	If <code>true</code> , it automatically the system does NOT wait for the response from the win transaction, so the balance returned will be after bet
<code>parallelRound</code>	<code>boolean</code>	OPTIONAL	<code>true</code>	If <code>true</code> , it will not assign <code>roundId</code> in system to allow play multiple rounds in parallel

Returns

Name	Type		Example	Description
<code>wager</code>	<code>object</code>	REQUIRED		See PlayWager
<code>balance</code>	<code>number</code>	OPTIONAL	<code>200.56</code>	Player's current balance in player's currency, sent only in initial play
<code>roundId</code>	<code>string</code>	REQUIRED	<code>c5b0c931-3971-4143-a478-15803f7837cc</code>	Round id
<code>complete?.finalWin</code>	<code>number</code>	OPTIONAL	<code>200.56</code>	Returned only if <code>complete</code> is <code>true</code> . Total win of the round (with potential win capping applied)
<code>complete?.balance</code>	<code>number</code>	OPTIONAL	<code>200.56</code>	Returned only if <code>complete</code> is <code>true</code> and <code>asyncWin</code> is <code>false</code> . Player's balance after win

Method: `complete` Async: YES Required: YES

```
const {balance} = await connector.complete();
```

Completes the round and pays the win to player's account.

Arguments

Name	Type		Example	Description
<code>asyncWin</code>	<code>boolean</code>	OPTIONAL	<code>true</code>	If <code>true</code> , it automatically the system does NOT wait for the response so returned balance will be based on last bet + <code>finalWin</code>

Returns

Name	Type		Example	Description
<code>balance</code>	<code>number</code>	OPTIONAL	<code>200.56</code>	Player's current balance in player's currency, sent only if <code>asyncWin</code> is <code>false</code>

Name	Type		Example	Description
finalWin	number	REQUIRED	200.56	Total win of the round (with potential win capping applied)

Method: info Async: YES Required: YES

```
const {config, state, bets, settings, betLimits} = await connector.info();
```

Returns game configuration and other information.

Arguments

Name	Type		Example	Description
roomId	string	OPTIONAL	df3d02ab-c6f5-4152-bbc3-9107e1f9bc7c	Room Id - only for multiplayer

Returns

Name	Type	Required	Example	Description
config	any	OPTIONAL	{paytable: {...}}	Game config
bets	any	OPTIONAL	{main: {...}, gamble: {...}}	Bet config per action. See BetConfig
state	any	OPTIONAL	{collection: 10}	Player's game state
settings	object	REQUIRED	{customDeposit: true, ...}	Player's currency
betLimits	object	OPTIONAL	{maxExposure: 10000, currencyRate: 1, ...}	Bet limits for game. See BetLimits

Method: recover Async: YES Required: YES

```
const unfinishedRound = await connector.recover();
```

Displays popup informing about unfinished rounds. If a player wishes to see the round it will return wagers of that round. If a player wishes to skip the round it will return new balance.

Arguments

Name	Type		Example	Description
skipPopup	boolean	OPTIONAL	true	Indicates if you want to skip popup and get raw data

Returns

Name	Type		Example	Description
.	Round	OPTIONAL	-	Unfinished round See Round
balance	number	OPTIONAL	100.24	Player's balance (only if players selects "SKIP" on popup)

Method: `gameFeed` Async: YES Required: NO

```
const {feed} = await connector.gameFeed(100);
```

Returns feed stored by the game. For example might represent roulette's hot numbers.

Arguments

Name	Type		Example	Description
amount	number	REQUIRED	3	Amount of recent wagers entries to return

Returns

Name	Type		Example	Description
feed	any[]	REQUIRED	[1, 3, 4]	Feed stored by the game, eg. hot numbers

Method: `cheats` Async: YES Required: YES

```
const {cheats} = await connector.cheats();
```

Lists available cheats. Available only in `development` mode.

Arguments None. Returns

Name	Type		Example	Description
cheats	string[]	REQUIRED	{main: ["win", "bonus"], bonus: ["retrigger"] }	Cheats available per <code>action</code>

Method: `replay` Async: YES Required: YES

```
const {round, currency, config, bets, state} = await connector.replay(roundId);
```

Displays popup with round summary and return it if a player wishes to watch it.

Arguments

Name	Type		Example	Description
roundId	string	REQUIRED	db76b227-0582-45bc-a2cd-fbf38449f28e	Round Id

Returns

Name	Type		Example	Description
{...}	ReplayRound	REQUIRED		See ReplayRound

Method: `getReplayUrl` Async: NO Required: NO

```
const url = await connector.getReplayUrl(roundId, game);
```

Returns URL for the replay

Arguments

Name	Type		Example	Description
roundId	string	REQUIRED	db76b227-0582-45bc-a2cd-fbf38449f28e	Round Id
game	string	REQUIRED	my-game	Game Id

Returns

Name	Type		Example	Description
.	string	REQUIRED	https://domain.com/launch/replay?roundId=XXX&game=YYY	Replay URL

Method: `balance` Async: YES Required: NO

```
const {balance} = await connector.balance();
```

Returns current player's balance

Arguments None.

Returns

Name	Type		Example	Description
balance	number	REQUIRED	123.45	Balance in player's currency

Method: `exit` Async: YES Required: YES


```
await connector.exit();
```

Displays popup confirming if player wants to quit the game and performs quitting.

Method: `isDepositSupported` Async: NO Required: NO

```
connector.isDepositSupported();
```

Returns information wheather opening despoit is supported.

Method: `openDeposit` Async: NO Required: NO

```
connector.openDeposit();
```

Opens deposit window.

Method: `reload` Async: YES Required: NO

```
await connector.reload();
```

Reloads game client.

Arguments None. Returns None.

Method: `gameHistory` Async: NO Required: NO

```
const show = replay => console.info("show replay", replay);
const hide = () => console.info("hide replay");
const close = () => console.info("popup closed");
connector.gameHistory(false, true, show, hide, close);
```

Shows the game history popup.

Arguments

Name	Type		Example	Description
<code>skipPopup</code>	<code>false</code>	OPTIONAL	-	Indicates if you want to skip popup and get raw data
<code>showWatch</code>	<code>true</code>	OPTIONAL	-	Indicates if you want to display "WATCH" button which enables

Name	Type		Example	Description
				replays
show	(replay:ReplayRound)=>null	OPTIONAL	-	Callback to open a replay of particular round. See ReplayRound
hide	()=>null	OPTIONAL	-	Callback to hide a replay of particular round
close	()=>null	OPTIONAL	-	Callback to close popup
page	number	OPTIONAL	-	Page of the game history (default 0)
skipReplayMode	boolean	OPTIONAL	-	If true it doesn't open replay mode inside the client. I.e. if watch button open new tab

Returns

Name	Type		Example	Description
.	GameHistoryRound[]	REQUIRED		See GameHistoryRound

Method: `initRealityCheck` Async: NO Required: NO

```
const onOpen = () => console.info("popup opened");
const onContinue = () => console.info("continue clicked");
const onGameHistory = () => connector.gameHistory();
connector.initRealityCheck(onOpen, onContinue, onGameHistory);
```

Initializes Reality Check window.

Arguments

Name	Type		Example	Description
onOpen	()=>any	OPTIONAL	-	Callback for opening the popup
onContinue	()=>any	OPTIONAL	-	Callback for pressing continue button.
onGameHistory	()=>any	OPTIONAL	-	Callback for pressing game history button. If not passed the button is not displayed

Returns None.

Method: `initPromoUI` Async: NO Required: NO

```
connector.initPromoUI();
```

Initializes Promo tools UI. Should be called after authentication and preferably after splash screen as it displays some UI.

Arguments None. **Returns** None.

Method: `setPromoUI` **Async:** NO **Required:** NO

```
connector.setPromoUI({x: 100, y: 100, alpha: 0.9, scale: 1.25});
```

Changes Promo UI's style - used to improve game logo visibility.

Arguments

Name	Type		Example	Description
<code>promoUIStyle</code>	PromoUIStyle	REQUIRED	-	See PromoUIStyle

Returns None.

Method: `setActiveBet` **Async:** YES **Required:** NO

```
await connector.setActiveBet(bet);
```

Informs Connector on the active bet value. Used by the Connector to inform players whether active bet qualifies for promo campaigns. Used by the Connector to inform integrated operators bridges on that event.

Arguments

Name	Type	Example	Description
<code>bet</code>	<code>number</code>	1.23	active bet value

Returns None.

Method: `gameLoaded` **Async:** NO **Required:** NO

```
await connector.gameLoaded();
```

Informs Connector when game assets finished loading. Used by the Connector to inform integrated operators when to start loading their modules and avoid slowing down game startup time.

Arguments None. **Returns** None.

Method: `muted` **Async:** NO **Required:** NO

```
await connector.muted();
```

Notifies Connector when player mutes the game. Used by the Connector to inform integrated operators bridges on that event.

Arguments None. **Returns** None.

Method: unmuted **Async:** NO **Required:** NO

```
await connector.unmuted();
```

Notifies Connector when player unmutes the game. Used by the Connector to inform integrated operators bridges on that event.

Arguments None. **Returns** None.

Method: turboToggled **Async:** NO **Required:** NO

```
await connector.turboToggled(value);
```

Notifies Connector when player toggles turbo capability. Used by the Connector to inform integrated operators bridges on that event.

Arguments

Name	Type	Example	Description
value	boolean	true	toggle value

Returns None.

Method: autoplayToggled **Async:** NO **Required:** NO

```
await connector.autoplayToggled(value);
```

Notifies Connector when player toggles autoplay capability. Used by the Connector to inform integrated operators bridges on that event.

Arguments

Name	Type	Example	Description
value	boolean	true	toggle value

Returns None.

Method: `paytableToggled` Async: NO Required: NO

```
await connector.paytableToggled(value);
```

Notifies Connector when player toggles payable capability. Used by the Connector to inform integrated operators bridges on that event.

Arguments

Name	Type	Example	Description
<code>value</code>	<code>boolean</code>	<code>true</code>	toggle value

Returns None.

Method: `helpToggled` Async: NO Required: NO

```
await connector.helpToggled(value);
```

Notifies Connector when player toggles help capability. Used by the Connector to inform integrated operators bridges on that event.

Arguments

Name	Type	Example	Description
<code>value</code>	<code>boolean</code>	<code>true</code>	toggle value

Returns None.

Method: `aboutToggled` Async: NO Required: NO

```
await connector.aboutToggled(value);
```

Notifies Connector when player toggles about capability. Used by the Connector to inform integrated operators bridges on that event.

Arguments

Name	Type	Example	Description
<code>value</code>	<code>boolean</code>	<code>true</code>	toggle value

Method: `rooms` Async: `YES` Required: `NO`

```
const rooms = await connector.rooms();
```

Returns rooms available for the player

Arguments No arguments.

Returns

Name	Type		Example	Description
<code>.</code>	<code>Room[]</code>	<code>REQUIRED</code>		Available rooms. See See Room

Method: `initMultiplayer` Async: `YES` Required: `NO`

```
const {send} = await connector.initMultiplayer(value);
send("command", {action: "main", bet, params: {multiplier}});
```

Method initializes connection for multiplayer games. It supports automatic reconnects. Supported types and payloads are game specific.

Arguments

Name	Type	Example	Description
<code>channel</code>	<code>string</code>	<code>abcd</code>	roomId
<code>onConnected</code>	<code>() => any</code>		handler triggered when connetion is established
<code>onDisconnected</code>	<code>() => any</code>		handler triggered when connection is closed
<code>onMessage</code>	<code>(message) => any</code>		handler triggered when message is received

Returns

Name	Type		Example	Description
<code>send</code>	<code>(type: "command" "cheat", payload: any) => void</code>	<code>REQUIRED</code>		Method used to send websocket messages

Types

ReplayRound

Name	Type		Example	Description
<code>send</code>	<code>Round</code>	<code>OPTIONAL</code>		See Round (only for single player games)

Name	Type		Example	Description
draw	any	OPTIONAL	{myReplayData:123}	Replay object (only for multiplayer games)
currency	string	REQUIRED	eur	Player's currency (internal platform code)
currencySymbol	string	REQUIRED	eur	Player's currency (to display to the player)
currencyDecimals	number	REQUIRED	2	Player's currency decimal places
bets	any	OPTIONAL	{main: {...}, gamble: {...}}	Bet config per action. See BetConfig
state	object	REQUIRED	{myState: 123}	Player's current state
config	number	REQUIRED	{myConfig: 123}	Game's config
settings	object	REQUIRED	{mySetting: "abc"}	Player's ettings
balanceAfter	number	REQUIRED	5.50	Balance affter round in player's currency

Round

Name	Type		Example	Description
roundId	string	REQUIRED	-	Round Id
wagers	Wager[]	REQUIRED	-	List of wagers. See Wager
previousState	any	OPTIONAL	{myState: 123}	Player's state from before the round

PlayWager

Name	Type		Example	Description
data	any	OPTIONAL	[{"spin": 1}, {"spin": 2}]	Game outcome
state	any	OPTIONAL	{"collection": 10}	Player's game state persistent between rounds
win	number	REQUIRED	16.45	Cash win in Player's currency
next	string[]	OPTIONAL	["gamble", "take"]	Next actions for multi-wager rounds. If response contains <code>next</code> array then in next play request <code>action</code> should be set to one of the array's elements.
feed	any	OPTIONAL	{hot:1}	Feed stored by the game, eg. hot numbers

Wager

Name	Type		Example	Description
createdAt	Date	REQUIRED	Mon Jan 29 2024 11:34:14 GMT+0100	Date of wager creation
data	any	OPTIONAL	{ "spin": 1 }, { "spin": 2 }	Game outcome
state	any	OPTIONAL	{ "collection": 10 }	Player's game state persistent between rounds
win	number	REQUIRED	16.45	Cash win in Player's currency
next	string[]	OPTIONAL	["gamble", "take"]	Next actions for multi-wager rounds. If response contains next array then in next play request action should be set to one of the array's elements.
action	string	REQUIRED	gamble	Action
bet	number	OPTIONAL	2.58	Cash bet in player's currency. It SHOULD NOT have more then 2 decimal places, otherwise it will be rounded using bankers rounding. Required only in the initial play
params	any	OPTIONAL	{ myValue: 123 }	Extra params passed to the game server

BetConfig

Name	Type		Example	Description
available	number[]	REQUIRED	[0.01, 0.5, 1, 5, 10]	Available bets in Player's currency
default	number	REQUIRED	0.5	Default bet in Player's currency
coin	number	REQUIRED	25	Coin

BetLimits

Name	Type		Example	Description
currencyRate	number	REQUIRED	1	Currency rate
exchangeRate	number	REQUIRED	1	Exchange rate
maxBet	number	REQUIRED	25	Maximum bet available
maxExposure	number	REQUIRED	25000	Max Exposure (payout) available
minBet	number	REQUIRED	1	Minimum bet available

GameHistoryRound

Name	Type		Example	Description
createdAt	Date	REQUIRED		Date of the round
roundId	string	REQUIRED		Player's currency
bet	number	REQUIRED	123.45	Bet in Player's currency
win	number	REQUIRED	123.45	Win in Player's currency
balanceAfter	number	OPTIONAL	123.45	Balance in Player's currency after the round
drawId	string	OPTIONAL		DrawId of the round for multiplayer games

PromoUIStyle

Name	Type		Example	Description
x	number	OPTIONAL	100	Horizontal position
y	number	OPTIONAL	100	Vertical position
alpha	number	OPTIONAL	0.9	Opacity
scale	number	OPTIONAL	1.25	Scale

Room

Name	Type		Example	Description
roomId	string	REQUIRED	abcd	Room Id
name	string	REQUIRED	my room	Name of the room
provider	string	REQUIRED	myProvide	Name of the provider
game	string	REQUIRED	myProvide	Name of the game
config	any	OPTIONAL	{delearName: "abc"}	Room config
connections	number	REQUIRED	50	Number of active connections to the room

Custom Promo UI

Below method should be used only if you want to create custom promo UI. In most cases you just skip this section

API

Method: `getCampaigns` Async: YES Required: NO

```
const campaigns = await connector.getCampaigns();
```

Gets all campaigns

Arguments None. Returns

Name	Type		Example	Description
.	Campaign[]	REQUIRED		See Campaign

Method: `getCampaign` Async: YES Required: NO

```
const campaign = await connector.getCampaign(campaignId, withTranslations);
```

Gets campaign details

Arguments

Name	Type	Example	Description
campaignId	string	db76b227-0582-45bc-a2cd-fbf38449f28e	Campaign Id
withTranslations	boolean	true	Fetches campaign translations

Returns

Name	Type		Example	Description
.	CampaignDetail	REQUIRED		See CampaignDetail

Method: `getActiveCampaignsInfo` Async: NO Required: NO

```
const campaignsInfo = await connector.getActiveCampaignsInfo();
```

Gets campaign info displayed in UI

Returns

Name	Type	Example	Description
freeBets	IFreeBetsCampaignInfo		See CampaignsInfo
prizeDrop	IPrizeDropCampaignInfo		See CampaignsInfo
tournament	ITournamentCampaignInfo		See CampaignsInfo

Method: `optCampaign` Async: YES Required: NO

```
await connector.optCampaign(campaignId, true);
```

Opts in/out from campaign

Arguments

Name	Type	Example	Description
campaignId	string	db76b227-0582-45bc-a2cd-fbf38449f28e	Campaign Id
optIn	boolean	true	Indicates if player opts in or out

Returns None.

Method: acknowledgeCampaign Async: YES Required: NO

```
await connector.acknowledgeCampaign(campaignId);
```

Acknowledges the campaign finish. After that the campaign will disappear from the list

Arguments

Name	Type	Example	Description
campaignId	string	db76b227-0582-45bc-a2cd-fbf38449f28e	Campaign Id

Returns None.

Method: playerEvent Async: YES Required: NO

```
await connector.playerEvent(campaignId, eventId, eventName, params);
```

Sends event to the promo tool

Arguments

Name	Type	Example	Description
campaignId	string	db76b227-0582-45bc-a2cd-fbf38449f28e	Campaign Id
eventId	string	db76b227-0582-45bc-a2cd-fbf38449f28e	Unique event id used for idempotency (same eventId will always return same response)
eventName	string	myCustomEvent	Event name to be handled by promo tool
params	string	myCustomEvent	Event payload object

Returns

Name	Type		Example	Description
.	any	OPTIONAL		Object returned by the promo tool

Method: `playerFeed` Async: YES Required: NO

```
await connector.playerFeed(campaignId, params);
```

Reads the value of the feed

Arguments

Name	Type	Example	Description
campaignId	string	db76b227-0582-45bc-a2cd-fbf38449f28e	Campaign Id
params	string	myCustomEvent	Event payload object

Returns

Name	Type		Example	Description
.	any	OPTIONAL		Object returned by the promo tool

Method: `campaignFeed` Async: YES Required: NO

```
await connector.campaignFeed(campaignId, params);
```

Reads the value of the feed

Arguments

Name	Type	Example	Description
campaignId	string	db76b227-0582-45bc-a2cd-fbf38449f28e	Campaign Id
params	string	myCustomEvent	Event payload object

Returns

Name	Type		Example	Description
.	any	OPTIONAL		Object returned by the promo tool

Types

Campaign

Name	Type		Example	Description
campaignId	string	REQUIRED	db76b227-0582-45bc-a2cd-fbf38449f28e	Campaign id
start	Date	OPTIONAL		Campaign start date
end	Date	OPTIONAL		Campaign end date
config	any	REQUIRED	{"abc": {...}}	Campaign config
type	string	REQUIRED	freeBets	Campaign type
name	string	REQUIRED	My campaign	Campaign name
status	string	REQUIRED	planned , started , active , finished	Campaign status

CampaignDetail

Name	Type		Example	Description
campaignId	string	REQUIRED	db76b227-0582-45bc-a2cd-fbf38449f28e	Campaign id
start	Date	OPTIONAL		Campaign start date
end	Date	OPTIONAL		Campaign end date
config	any	REQUIRED	{"abc": {...}}	Campaign config
type	string	REQUIRED	freeBets	Campaign type
type	name	REQUIRED	My campaign	Campaign name
status	string	REQUIRED	planned , started , active , finished	Campaign status
campaignState	any	REQUIRED	{"abc": {...}}	Campaign state
playerState	any	REQUIRED	{"abc": {...}}	Campaign player state

CampaignsInfo

Name	Type		Example	Description
freeBets	IFreeBetsCampaignInfo	OPTIONAL	{"used": 5, "total": 10}	FreeBets info
prizeDrop	IPrizeDropCampaignInfo	OPTIONAL	{"left": 5, "total": 10}	PrizeDrop info
tournament	ITournamentCampaignInfo	OPTIONAL	{"playerRanked": 2, "totalPositions": 10}	Tournament info

Provably Fair API

Below method should be used to implement Provably Fair UI

API

Method: `activeRngSeeds` Async: YES Required: NO

```
const rngSeeds = await connector.activeRngSeeds();
```

Returns active seeds

Arguments None. Returns

Name	Type		Example	Description
.	ProvablyFairSeeds	REQUIRED		See ProvablyFairSeeds

Method: `updateClientSeed` Async: YES Required: NO

```
const rngSeeds = await connector.updateClientSeed(clientSeed);
```

Sets new client seeds and rotates server seed

Arguments

Name	Type	Example	Description
clientSeed	string	abcd	New client seed

Returns

Name	Type		Example	Description
.	ProvablyFairSeeds	REQUIRED		See ProvablyFairSeeds

Method: `roundRngState` Async: YES Required: NO

```
const roundState = await connector.roundRngState(roundId);
```

Returns provably fair details of the round

Arguments

Name	Type	Example	Description
roundId	string	db76b227-0582-45bc-a2cd-fbf38449f28e	Round Id

Returns

Name	Type		Example	Description
.	ProvablyFairRoundState	REQUIRED		See ProvablyFairRoundState

Method: `unhashServerSeed` Async: YES Required: NO

```
const serverSeed = await connector.unhashServerSeed(serverSeedHash);
```

Returns server seed based on its hashed version

Arguments

Name	Type	Example	Description
serverSeedHash	string	abcdabcdabcdabcdabcdabcdabcdabcdabcdabcdabcdabcdabcdabcdabcdabcd	Hashed server see

Returns

Name	Type		Example
serverSeed	string	REQUIRED	abcdabcdabcdabcdabcdabcdabcdabcdabcdabcdabcdabcdabcdabcdabcdabcd

Method: `drawRngState` Async: YES Required: NO

```
const drawState = await connector.drawRngState(drawId);
```

Returns provably fair details of the multiplayer draw

Arguments

Name	Type	Example	Description
drawId	string	db76b227-0582-45bc-a2cd-fbf38449f28e	Draw Id

Returns

Name	Type		Example	Description
.	ProvablyFairDrawState	REQUIRED		See ProvablyFairDrawState

Method: `proveFairness` Async: YES Required: NO

```
const fairnessProof = await connector.proveFairness({serverSeed, clientSeed, nonce}, {numberOfMines: 10});
```

```
const multiplayerFairnessProof = await connector.proveFairness({hash, seed}, {rtp: 0.97, maxMultiplier: 10000});
```

Method supports both single-player and multiplayer provable fairness. Returns proof of the fair gameplay for a given `seeds` and `nonce`, or given `seed` and `hash` in multiplayer case.

Arguments

Name	Type		Description
<code>rngState</code>	<code>RoundRngState</code> or <code>DrawRngState</code>	REQUIRED	server seed
<code>data</code>	<code>any</code>	OPTIONAL	data specific for the game, to allow precise proof

Returns

Name	Type		Example	Description
.	<code>FairnessProof</code>	REQUIRED		See FairnessProof

Types

RoundRngState

Name	Type	Example	Description
<code>serverSeed</code>	<code>string</code>	<code>69742f105765e8d0a35a5819918a8d945f2a8f9f211e60c308a83c2af3a2c759</code>	server seed
<code>clientSeed</code>	<code>string</code>	<code>my-client-seed</code>	client seed
<code>nonce</code>	<code>number</code>	<code>13</code>	nonce

DrawRngState

Name	Type	Example	Description
<code>hash</code>	<code>string</code>	<code>69742f105765e8d0a35a5819918a8d945f2a8f9f211e60c308a83c2af3a2c759</code>	server hash
<code>seed</code>	<code>string</code>	<code>players-seed</code>	players' seed

ProvablyFairSeeds

Name	Type		Example	Description
<code>clientSeed</code>	<code>string</code>	REQUIRED	<code>abcd</code>	Client seed
<code>serverSeedHash</code>	<code>string</code>	REQUIRED	<code>abcd</code>	Hashed server seed
<code>nextServerSeedHash</code>	<code>string</code>	REQUIRED	<code>abcd</code>	Hashed next server seed
<code>nonce</code>	<code>number</code>	REQUIRED	<code>0</code>	nonce
<code>unfinishedGames</code>	<code>string[]</code>	REQUIRED	<code>["game1", "game2"]</code>	List of games with unfinished rounds

ProvablyFairRoundState

Name	Type		Example	Description
clientSeed	string	REQUIRED	abcd	Client seed
serverSeedHash	string	REQUIRED	abcd	Hashed server seed
nextServerSeedHash	string	REQUIRED	abcd	Hashed next server seed
nonce	number	REQUIRED	0	nonce
serverSeed	string	OPTIONAL	abcd	Server seed
status	"active" or "revealed"	REQUIRED	active	Status of the server seed

ProvablyFairDrawState

Name	Type		Example	Description
hash	string	REQUIRED	abcd	Current server hash
seed	string	REQUIRED	abcd	Players' seed for the given room
hashIndex	string	REQUIRED	12342	Index of the hash in the hash chain

FairnessProof

Name	Type		Example	Description
hashes	Hash[]	REQUIRED		List of hashes generated over the course of the given round, to extract random numbers. See Hash
randomizations	Randomization[]	REQUIRED		List of randomizations requested by the game for a given round. See Randomization

Hash

Name	Type		Example	Description
hex	string	REQUIRED	634ce936de372e6033083cbce378edaa53734ac19f69ebc193cf44fe5dcb29fc	Hex representation of the hash
bytes	number[]	REQUIRED	[99, 76, 233, 54, 222, 55, 46, 96, 51, 8, 60, 188, 227, 120, 237...]	Array of bytes representing the hash

Randomization

Name	Type		Example	Description
<code>limit</code>	<code>number</code>	REQUIRED	<code>10001</code>	Limit requested by the game for a given randomization, i.e. <code>random(limit)</code> call made by the game
<code>extractions</code>	<code>Extraction[]</code>	REQUIRED		A list of attempts to get the integer satisfying limit condition, as in Unbiased Integer Randomisation Algorithm used in the system. See Extraction
<code>randomNumber</code>	<code>number</code>	REQUIRED	<code>2753</code>	Resulting random number
<code>gameEvent</code>	<code>any</code>	REQUIRED	<code>{roll: 27.53}</code>	Game specific result that is generated based on the current randomization

Extraction

Name	Type		Example	Description
<code>cursor</code>	<code>number</code>	REQUIRED	<code>4</code>	Cursor value for a given extraction
<code>hashIndex</code>	<code>number</code>	REQUIRED	<code>0</code>	Equal to <code>Math.floor(cursor / 8)</code> - index of a hash from which the current extraction is performed; refers to the <code>hashes</code> list
<code>offset</code>	<code>number</code>	REQUIRED	<code>4</code>	Equal to <code>cursor % 8</code> - offset within a current hash, which points to the integer being extracted
<code>integer</code>	<code>number</code>	REQUIRED	<code>1400064705</code>	Integer returned by the extraction for Unbiased Integer Randomization algorithm. This integer might still be rejected by the algorithm, resulting in next extraction attempt

Currencies formatting

Below methods should be used to support currencies with varying decimals and games with floating point payouts

API

Method: `formatCurrency` Async: NO Required: NO

```
const formattedString = await connector.formatCurrency(0.0000001289, "btc", "en-GB");
```

Floors the value to the decimal precision specified by the platform for a given currency and formats it for display.

Arguments

Name	Type		Example	Description
<code>value</code>	<code>number</code>	REQUIRED	<code>0.0000001289</code>	value to be formatted

Name	Type		Example	Description
currency	string	OPTIONAL	"btc"	currency, defaults to player currency
language	string	OPTIONAL	"en-GB"	language, defaults to player language

Returns

Name	Type		Example	Description
N/A	string	REQUIRED	BTC 0.00000012	floored and formatted string to display

Method: `fetchCurrencyExchangeRates` **Async:** YES **Required:** NO (required if game requires base currency conversions)

```
await connector.fetchCurrencyExchangeRates();
```

Loads all currencies exchange rates from the server - it's required for the game to get base currency values (e.g. for leaderboard sorting purpose)

Arguments None. **Returns**

Name	Type		Example	Description
N/A	{[key:string]: number}	REQUIRED	{"btc": 8}	Decimals for each currency

Method: `getExchangeRate` **Async:** NO **Required:** NO

```
const decimals = await connector.getExchangeRate("btc");
```

Returns decimals number for a given currency.

Arguments

Name	Type		Example	Description
currency	string	OPTIONAL	"btc"	currency, defaults to the players currency

Returns

Name	Type		Example	Description
N/A	number	REQUIRED	8	exchange rates for the given currency

Method: `convertToBaseCurrency` **Async:** NO **Required:** NO

```
const baseCurrencyValue = await connector.convertToBaseCurrency(0.0000001289, "btc");
```

Converts value in a given currency to the base currency value.

Arguments

Name	Type		Example	Description
value	number	REQUIRED	0.0001289	value to be converted
currency	string	OPTIONAL	btc	currency, defaults to player currency

Returns

Name	Type		Example	Description
N/A	number	REQUIRED	9.552019779	value converted to the base currency

Method: fetchCurrencyDecimals **Async:** YES **Required:** NO (required if multiple currencies require formatting during the gameplay)

```
await connector.fetchCurrencyDecimals();
```

Loads all currencies decimals from the server - it's required for the game to display currencies different then the player's default one.

Arguments None. Returns

Name	Type		Example	Description
N/A	{[key:string]: number}	REQUIRED	{"btc": 8}	Decimals for each currency

Method: getCurrencyDecimal **Async:** NO **Required:** NO

```
const decimals = await connector.getCurrencyDecimal("btc");
```

Returns decimals number for a given currency.

Arguments

Name	Type		Example	Description
currency	string	OPTIONAL	"btc"	currency, defaults to the players currency

Returns

Name	Type		Example	Description
N/A	number	REQUIRED	8	Decimals for the given currency

Method: showLobbyButton Async: NO Required: NO

```
const showLobbyButton = await connector.showLobbyButton();
```

Returns boolean value based on lobbyUrl existence.

Arguments None. Returns

Name	Type		Example	Description
N/A	boolean	REQUIRED	false	Boolean value for showing lobby button

Method: getChannel Async: NO Required: NO

```
const channel = await connector.getChannel();
```

Returns channel value passed in URL.

Arguments None. Returns

Name	Type		Example	Description
N/A	string	REQUIRED	desktop	String value indicates if channel is desktop or mobile

Method: floor Async: NO Required: NO

```
const flooredValue = await connector.floor(0.0000001289, 8);
```

Floors the given value to the specified number of decimals after the coma.

Arguments

Name	Type		Example	Description
value	number	REQUIRED	0.0000001289	value to be floored
decimals	number	REQUIRED	8	number of decimal places after the coma

Returns

Name	Type		Example	Description
N/A	number	REQUIRED	0.00000012	value floored to the given amount of decimals

Method: floorCurrency Async: NO Required: NO

```
const flooredValue = await connector.floorCurrency(0.0000001289, "btc");
```

Floors the given value to number of decimals specific for a given currency.

Arguments

Name	Type		Example	Description
value	number	REQUIRED	0.0000001289	value to be floored
currency	string	OPTIONAL	btc	currency, defaults to player currency

Returns

Name	Type		Example	Description
N/A	number	REQUIRED	0.00000012	value floored to the number of decimals specific currency